

سیستم عامل

دانشگاه فردوسی مشهد | گروه مهندسی کامپیوتر

استاد: دکتر اله‌بخش

فاز ۱ از ۵ | وزن ۱۰٪ از نمره‌ی پروژه

First Contact

فرآیندها، PCB و ارتباط بین‌فرآیندی

ایستگاه ECHO-9 بیدار شده است و فقط یک در ورودی دارد.

Processes ▷ PCB & Lifecycle ▷ AF_UNIX IPC

#	نام و نام‌خانوادگی	آیدی
۱	حانیه ملک‌جعفریان	@HaniehMalekjafarian
۲	آرمان بیجاری	@ArmanBjr

فهرست مطالب

۱ ECHO-9 Station ۲
۱.۱ AgentOS چیست؟ ۲
۲.۱ مسیر پروژه ۳
۳.۱ مسیر یک درخواست ۴

۲ پیاده‌سازی AgentOS در فاز اول ۵

۳ راه‌اندازی محیط پروژه ۶
۱.۳ چیزهایی که باید نصب داشته باشین ۷
۲.۳ گرفتن پروژه با Git ۷
۳.۳ ورود به محیط Ubuntu و ساخت پروژه ۸
۴.۳ راه‌اندازی Ollama و ایستگاه ۸
۵.۳ چطور بفهمیم کارمون درسته؟ ۹

۴ تحویل با Git ۹
۱.۴ شاخه‌ها و Pull Request ۹
۲.۴ تست خودکار با GitHub Actions ۱۱
۳.۴ گرفتن تغییرهای فاز بعد ۱۲

۵ ایستگاه واقعی و رابط‌های کاربری ۱۲
۱.۵ Mission Control ۱۲
۲.۵ Process Monitor ۱۳
۳.۵ Protocol Workbench ۱۴
۴.۵ Live Station View ۱۵
۵.۵ چرا این رابط‌ها مهمن؟ ۱۶

۶ چیزهایی که باید تحویل بدین ۱۷
۱.۶ مخزن خصوصی گروه ۱۷
۲.۶ گزارش روند انجام پروژه ۱۷
۳.۶ اظهارنامه‌ی استفاده از هوش مصنوعی ۱۸

۷ سیاست استفاده از هوش مصنوعی ۱۸
۱.۷ استفاده‌های مجاز ۱۸
۲.۷ استفاده‌های غیرمجاز ۱۸
۳.۷ شفافیت استفاده از هوش مصنوعی ۱۹
۴.۷ ارزیابی و روز ارائه ۱۹

۱ ECHO-9 Station

ECHO-9 یک ایستگاه فضاییه که ساکنین اون ایجنت‌ها هستن. هر کدوم از این ایجنت‌ها توی بخش‌های مختلف ایستگاه وظایف مشخصی دارن و برای انجام کارهاشون می‌تونن از هسته‌ی هوش مصنوعی که در مرکز ایستگاه قرار داره کمک بگیرن. برای مثال، ایجنت‌ها می‌تونن درخواست‌هاشون رو از طریق رابط Mission Control ارسال کنن. این درخواست‌ها از طریق یک واسطه‌ی پایتونی و Unix Socket به سرور AgentOS می‌رسن و بعد از بررسی توسط پارسر و Dispatcher، در نهایت به هسته‌ی هوش مصنوعی فرستاده می‌شن. جواب تولید می‌شه و همین مسیر رو برمی‌گرده تا به ایجنت برسه. البته درخواست‌ها لزوماً نباید از Mission Control ارسال بشن و کلاینت‌ها می‌تونن مستقیماً هم از طریق Unix Socket با سرور ارتباط برقرار کنن.

توی همین ارسال و دریافت پاسخ ساده، فرض کنین ایجنتی شروع به ارسال پیام می‌کنه اما به دلایلی هیچ‌وقت اون رو به پایان نمی‌رسونه. چه اتفاقی می‌افته؟ یا چی می‌شه اگه کسی پیامی طولانی‌تر از اندازه‌ی بافر ارسال کنه؟ اصلاً از کجا می‌فهمیم چه ایجنت‌هایی توی ایستگاه حضور دارن؟ اگه یکی از ایجنت‌ها وسط انجام کارش متوقف بشه چی؟ حالا فرض کنین این ایستگاه کلی منبع مختلف داره و یک عالمه ایجنت که باید کارهایی که بهشون assign شده رو انجام بدن. به نظرتون اولویت تمام کارها توی ایستگاه برابره؟ طبیعتاً نه. چجوری باید این اولویت‌ها رو توی سیستم رعایت کنیم؟ اگه چند ایجنت هم‌زمان به یک منبع مشترک نیاز داشته باشن چی؟

این‌ها فقط چند نمونه از مسائلی هستن که برای اداره‌ی چنین ایستگاهی باید براشون راه‌حل داشته باشیم.

۱.۱ AgentOS چیست؟

برای مدیریت این شرایط و مشکلاتی که ممکنه توی ایستگاه فضایی پیش بیاد، نیاز به یک سیستم مدیریتی داریم که ما بهش می‌گیم AgentOS.

این سیستم مسئول مدیریت فعالیت ایجنت‌هاست؛ از شناسایی اون‌ها و مدیریت چرخه‌ی عمرشون گرفته تا سازمان‌دهی اجرای کارها، تخصیص منابع، هماهنگی بین بخش‌های مختلف و مدیریت شرایطی که همه‌چیز طبق انتظار پیش نمی‌ره. وظیفه‌ی شما در این پروژه، طراحی و توسعه‌ی این سیستمه.

البته قرار نیست از همون ابتدا تمام این قابلیت‌ها رو پیاده‌سازی کنین. AgentOS توی ۵ فاز مختلف توسعه پیدا می‌کنه و در هر فاز، بخش جدیدی به همون سیستمی که از قبل ساختین اضافه می‌شه.

از اینجا به بعد، تصمیم‌هایی که در طراحی AgentOS می‌گیرین، فقط روی نتیجه‌ی یک تست یا اجرای یک برنامه تأثیر نمی‌ذارن؛ این تصمیم‌ها تعیین می‌کنن ایستگاه ECHO-9 چجوری کار کنه، در برابر مشکلات چه واکنشی نشون بده و تا چه اندازه بتونه به فعالیتش ادامه بده.

AgentOS یک سرویسه که توی فضای کاربر (User Space) و با زبان C نوشته می‌شه. توی این پروژه قرار نیست کرنل لینوکس رو تغییر بدین یا یک سیستم‌عامل کامل از صفر بسازین. بخش‌هایی مثل مدل زبانی، رابط وب، پارسر JSON و کلاینت HTTP هم از قبل آماده شدن و در اختیارتون قرار می‌گیرن تا تمرکز اصلی روی پیاده‌سازی و درک مفاهیم سیستم‌عامل باشه.

> **یک قدم فراتر از ECHO-9...**
> اگه هنوز تصویر کلی چیزی که قراره بسازیم براتون روشن نیست، یا کنجکاوین بدونین ایده‌ی مدیریت ایجنت‌های هوش مصنوعی در دنیای واقعی چطور دنبال می‌شه، این منابع می‌تونن براتون جالب باشن.
>
> - **برای شروع، یک ویدئوی کوتاه:** Why AI Agents Need an Operating System – IBM Technology
>
> این ویدئو توضیح می‌ده چرا با بیشتر شدن تعداد ایجنت‌ها، صرفاً داشتن مدل‌های هوشمند کافی نیست و به زیرساختی برای مدیریت اون‌ها نیاز داریم.

- **برای آشنایی با یک پروژه‌ی تحقیقاتی واقعی:** AIOS: LLM Agent Operating System
  این مقاله یک معماری بزرگ‌تر برای مدیریت ایجنت‌های مبتنی بر مدل‌های زبانی معرفی می‌کنه. مفاهیمی مثل زمان‌بندی، مدیریت حافظه، مدیریت منابع و کنترل دسترسی که در طول پروژه باهاشون سروکار داریم، در این پژوهش هم مطرح شدن.
- **برای دیدن یک نمونه‌ی کاربردی:** Designing Grok Bot for a World of Persistent Agents
  این مطلب نشون می‌ده چطور ایجنت‌هایی با هویت، حافظه و وظایف مستقل می‌تونن کنار هم کار کنن. تمرکزش بیشتر روی طراحی و هماهنگی ایجنت‌هاست، نه پیاده‌سازی مستقیم مفاهیم سیستم‌عامل.
- **برای علاقه‌مندها به مدیریت حافظه:** MemGPT: Towards LLMs as Operating Systems
  این پژوهش نشون می‌ده چطور می‌شه از ایده‌های مدیریت حافظه در سیستم‌عامل‌ها برای مدیریت حافظه و زمینه‌ی متنی مدل‌های زبانی الهام گرفت.

۲.۱. مسیر پروژه

قراره AgentOS رو توی پنج فاز مختلف توسعه بدیم. هر فاز بخش جدیدی به سیستم اضافه می‌کنه و در نهایت تمام این بخش‌ها باید در کنار هم کار کنن.

در فاز اول، ارتباطات بین‌فرآیندی و مدیریت منطقی فرآیندها رو پایه‌گذاری می‌کنیم. در فاز دوم، سراغ نخ‌ها، اجرای موازی ایجنت‌ها و همگام‌سازی دسترسی به منابع مشترک می‌ریم. فاز سوم به الگوریتم‌های زمان‌بندی، اولویت‌بندی، مهلت‌های زمانی و سیاست‌های چندهسته‌ای اختصاص داره. در فاز چهارم، مسئله‌ی بن‌بست، تخصیص منابع و روش‌های پیشگیری، اجتناب، شناسایی ددلاک و بازیابی رو بررسی می‌کنیم. در نهایت، فاز پنجم به مدیریت حافظه، صفحه‌بندی، جایگزینی صفحه و بازیابی سیستم پس از خرابی می‌پردازه.

جدول زیر یک دید کلی از مسیر پروژه، موضوعات هر فاز و زمان‌بندی اون‌ها رو نشون می‌ده.

فاز	بازه‌ی زمانی	موضوعات اصلی	وزن
P1	۱۰ آبان تا ۲۴ آبان	فرآیندها، ارتباط بین‌فرآیندی (IPC)، مدیریت PCB و چرخه‌ی عمر فرآیند، پروتکل Socket و ارتباط با هسته‌ی هوش مصنوعی	۱۰٪
P2	۲۷ آبان تا ۱۱ آذر	نخ‌ها و اجرای موازی ایجنت‌ها، همگام‌سازی با Mutex و Semaphore، بافرهای محدود، مسئله‌ی Readers-Writers و متغیرهای شرطی (Condition Variables)	۲۰٪
P3	۱۵ آذر تا۲۹ آذر	شش الگوریتم زمان‌بندی CPU، پیش‌دستی مشارکتی (Cooperative Preemption)، معیارهای ارزیابی، مهلت‌های زمانی (Deadlines) و مقایسه‌ی زمان‌بندی چندهسته‌ای منطقی	۲۵٪
P4	۲ دی تا ۱۶ دی	تخصیص منابع و بن‌بست (Deadlock)، پیشگیری، اجتناب با الگوریتم بانکدار (Banker's Algorithm) و روش‌های گرافی، شناسایی بن‌بست و بازیابی	۲۵٪
P5	۲۰ دی تا ۱۸ بهمن	تخصیص حافظه، قطعه‌بندی (Segmentation)، صفحه‌بندی (Paging)، TLB، الگوریتم‌های جایگزینی صفحه، ماندگاری داده‌ها (Persistence) و بازیابی پس از خرابی	۲۰٪

جدول ۱: زمان‌بندی و موضوعات پنج فاز پروژه‌ی AgentOS

قانون تحویل با تأخیر

برای هر فاز، تا **۷۲ ساعت (۳ روز)** بعد از ددلاین اصلی امکان تحویل پروژه با کسر نمره وجود داره. میزان جریمه بر اساس مدت تأخیر به شکل زیر محاسبه می‌شه:

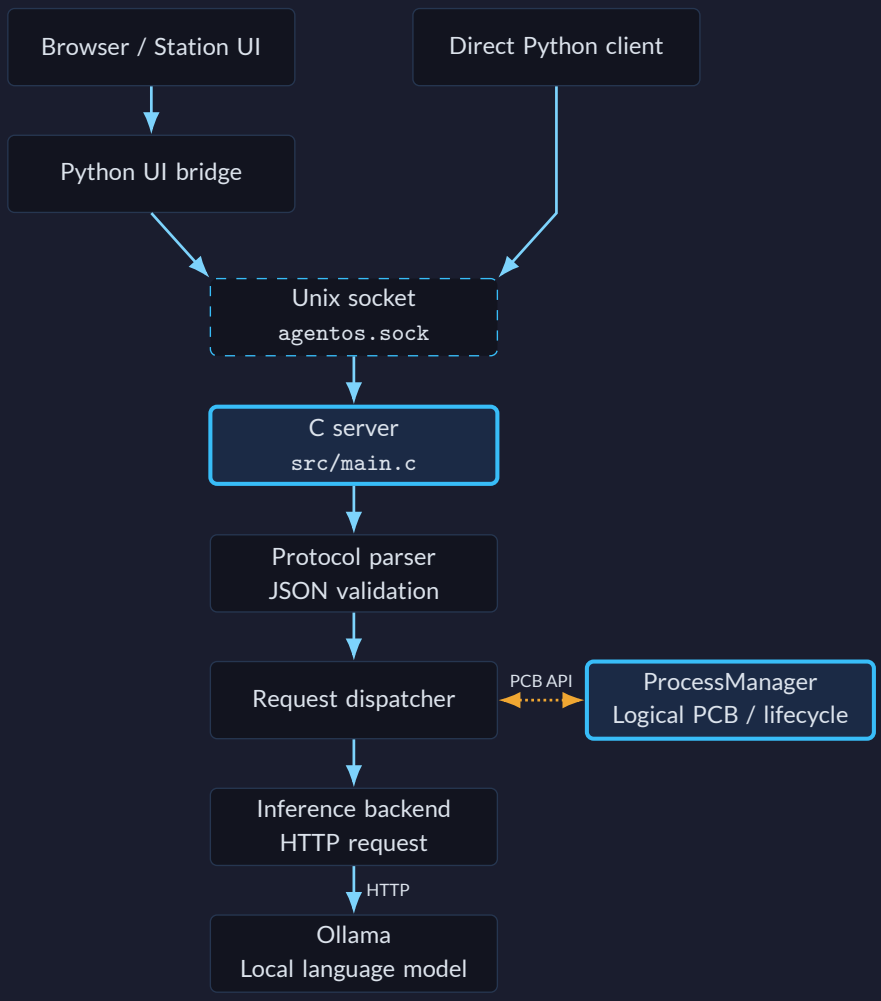
مدت تأخیر	۰ تا ۱۲ ساعت	۱۲ تا ۲۴ ساعت	۲۴ تا ۴۸ ساعت	۴۸ تا ۷۲ ساعت
ضریب تأخیر	۳٪	۵٪	۱۰٪	۲۰٪

توجه: درصدهای بالا از نمره‌ی کسب‌شده کسر می‌شن. بعد از گذشت ۷۲ ساعت از ددلاین اصلی، تحویل اون فاز پذیرفته نمی‌شه. این قانون برای تمام فازهای پروژه اعمال می‌شه.

لطفا توجه کنین که برای کسب نمره کامل از پروژه باید حداقل ۵۰٪ نمره هر فاز را کسب کنین و نمرات اضافه تعریف شده در طول پروژه جایگزین فازهای اصلی پروژه نخواهند بود.

۳.۱. مسیر یک درخواست

مسیر کلی یک درخواست از ایجنت تا هسته‌ی هوش مصنوعی ایستگاه در شکل زیر نمایش داده شده.



شکل ۱: مسیر کلی درخواست ایجنت تا مدل زبانی محلی.

همون‌طور که می‌بینین، درخواست‌ها می‌تونن از دو مسیر وارد سیستم بشن. توی مسیر اول، درخواست از طریق رابط وب (Mission Control) ارسال می‌شه و یک واسطه‌ی پایتونی (Python UI Bridge) اون رو به سرور منتقل می‌کنه. اما توی مسیر دوم، یک کلاینت پایتونی می‌تونه بدون نیاز به رابط وب، مستقیماً از طریق Unix Socket با سرور ارتباط برقرار کنه. در هر دو حالت، درخواست در نهایت به سروری می‌رسه که با زبان C نوشته شده. این سرور مسئول برقراری ارتباط با کلاینت‌ها، دریافت درخواست‌ها و برگردوندن پاسخ‌هاست.

بعد از دریافت یک درخواست کامل، پیام به Protocol Parser داده می‌شه تا ساختار و اعتبار اون بررسی بشه. برای مثال، آیا پیام یک JSON معتبره؟ آیا فیلدهای ضروری رو داره؟ آیا عملیات درخواست‌شده توسط سیستم پشتیبانی می‌شه؟

اگه درخواست معتبر باشه، به Dispatcher می‌رسه. اینجا علاوه بر اینکه درخواست برای پردازش آماده می‌شه، باید وضعیت ایجنت فرستنده هم مشخص باشه. آیا این ایجنت قبلاً توی سیستم ثبت شده؟ آیا هنوز فعاله؟ یا قبلاً از سیستم حذف شده و دیگه اجازه‌ی ارسال درخواست نداره؟

برای مدیریت این اطلاعات، از بخشی به اسم ProcessManager استفاده می‌کنیم. این بخش اطلاعات ایجنت‌ها و وضعیت اون‌ها رو نگه می‌داره و Dispatcher در زمان لازم توابع اون رو صدا می‌زنه. پس ProcessManager یک مرحله‌ی جداگانه برای انتقال پیام نیست، بلکه بخشی از مدیریت داخلی AgentOS محسوب می‌شه. بعد از انجام بررسی‌های لازم، درخواست از طریق Inference Backend و با استفاده از HTTP به Ollama فرستاده می‌شه. مدل زبانی پاسخ رو تولید می‌کنه و نتیجه از مسیر سرور به کلاینت برمی‌گرده.

نکته اینجاست که کل این مسیر باید حتی وقتی درخواست‌ها ناقص یا نامعتبر هستن هم رفتار مشخصی داشته باشه. مثلاً یک درخواست نامعتبر نباید باعث ثبت شدن یک ایجنت جدید یا به‌هم‌ریختن وضعیت بقیه‌ی ایجنت‌ها بشه.

۲ پیاده‌سازی AgentOS در فاز اول

توی این فاز پروژه فقط قراره **دو کادری که با رنگ آبی روشن در شکل ۱ مشخص شدن، رو تکمیل و پیاده‌سازی کنین.**

- `src/main.c` — **C Server**

  این بخش مسئول ارتباط بین کلاینت‌ها و AgentOS هست.

  باید سروری رو کامل کنین که بتونه از طریق Unix Socket به درخواست‌های کلاینت‌ها گوش بده، اتصال‌ها رو بپذیره، پیام‌ها رو درست دریافت کنه و بعد از پردازش، پاسخ رو به‌صورت کامل به فرستنده برگردونه.

  توی این فایل، چهار قسمت با شناسه‌های M1.1 تا M1.4 برای پیاده‌سازی شما مشخص شده:

  - M1.1 — **راه‌اندازی سرور:** یک نقطه‌ی ارتباطی واقعی با Unix Socket بسازین و سرور رو آماده‌ی پذیرش اتصال کنین.
  - M1.2 — **ارسال قابل اعتماد:** مطمئن بشین پاسخ به‌صورت کامل ارسال می‌شه.
  - M1.3 — **مدیریت یک درخواست:** پیام رو به‌صورت کامل دریافت کنین، برای بررسی به بخش آماده‌ی پروتکل تحویل بدین و حتی در شرایطی مثل درخواست ناقص یا نامعتبر، رفتار مشخصی داشته باشین.
  - M1.4 — **ادامه‌ی فعالیت سرور:** اتصال‌ها رو یکی‌یکی مدیریت کنین؛ با تموم شدن یک ارتباط، سرور نباید متوقف بشه و باید برای درخواست بعدی آماده بمونه.

> توی این فاز سرور به‌صورت Sequential کار می‌کنه و قرار نیست برای پردازش هم‌زمان درخواست‌ها از Thread استفاده کنین.

- **ProcessManager — `src/process_manager.c`**

این بخش مسئول نگهداری اطلاعات ایجنت‌ها و مدیریت چرخه‌ی عمر اون‌هاست.
هر ایجنتی که توی سیستم ثبت می‌شه، یک شناسه‌ی منطقی به اسم PID دریافت می‌کنه و اطلاعاتش توی ساختاری به اسم PCB نگهداری می‌شه.
البته این PID با شناسه‌ی فرآیندهای واقعی لینوکس فرق داره. فعلاً داریم یک مدل منطقی از فرآیندها و چرخه‌ی عمرشون رو داخل AgentOS پیاده‌سازی می‌کنیم، نه اینکه برای هر ایجنت یک فرآیند واقعی لینوکسی بسازیم.
شما باید سازوکاری رو پیاده‌سازی کنین که بتونه ایجنت‌های جدید رو ثبت کنه، اطلاعات اون‌ها رو نگه داره، تغییر وضعیتشون رو مدیریت کنه و در صورت خاتمه‌ی فعالیت، اطلاعات لازم رو تا زمان جمع‌آوری نهایی نگه داره.
همچنین باید امکان حذف، غیرفعال کردن و فعال‌سازی مجدد ایجنت‌ها وجود داشته باشه.
توی این فایل، بخش‌های P1.5 تا P1.9 به عهده‌ی شماست.

- **P1.5 — مدیریت داده‌ها:** ساختار مدیریتی ProcessManager رو ایجاد کنین، اطلاعات ایجنت‌ها و هویت‌های غیرفعال رو نگه دارین و در نهایت منابعش رو به‌درستی آزاد کنین.
- **P1.6 — ثبت و شناسایی:** ایجنت‌های جدید رو ثبت کنین، به هر کدوم یک PID منطقی اختصاص بدین و امکان جست‌وجو و دریافت اطلاعات PCB رو فراهم کنین. همچنین باید از ثبت هویت‌های تکراری جلوگیری کنین.
- **P1.7 — مدیریت وضعیت:** چرخه‌ی عمر ایجنت‌ها رو با پنج حالت NEW، READY، RUNNING، WAITING و TERMINATED مدیریت کنین. فقط تغییر وضعیت‌های مجاز باید پذیرفته بشن و درخواست‌های نامعتبر نباید وضعیت سیستم رو تغییر بدن.
- **P1.8 — خاتمه و جمع‌آوری:** وقتی فعالیت یک ایجنت تموم می‌شه، اطلاعاتش نباید بلافاصله حذف بشه. باید تا زمان جمع‌آوری نهایی (Reap) به‌صورت Zombie باقی بمونه و بعد فقط یک بار جمع‌آوری بشه.
- **P1.9 — مدیریت جمعیت:** امکان اضافه کردن، غیرفعال کردن و فعال‌سازی مجدد هویت ایجنت‌ها رو فراهم کنین، بدون اینکه شناسه‌های منطقی قبلی دوباره مورد استفاده قرار بگیرن.

> **محدوده‌ی پیاده‌سازی شما**
> در این فاز فقط باید قسمت‌های مشخص‌شده در src/main.c و `scr/process_manager.c` رو کامل کنین.
> بقیه‌ی کدهای آماده‌ی پروژه، از جمله Protocol Parser، Dispatcher، Inference Backend، Control Plane و رابط‌های نمایشی، از قبل پیاده‌سازی شدن.
> نباید کدهای این بخش‌ها، فایل‌های Header، تست‌های ارائه‌شده یا تنظیمات اصلی پروژه رو تغییر بدین.
> به‌خصوص فایل `include/process_manager.h` قرارداد مشخصی بین کد شما و بقیه‌ی سیستمه. باید توابع تعریف‌شده در اون رو پیاده‌سازی کنین، نه اینکه امضای توابع یا ساختارهای تعریف‌شده رو تغییر بدین.

در نهایت، با کامل شدن این دو بخش، درخواست‌های ایجنت‌ها می‌تونن از طریق سرور واقعی شما به مدل زبانی برسن و در کنار اون، اطلاعات و تغییر وضعیت ایجنت‌ها هم توسط ProcessManager خودتون مدیریت بشه. این همون زیرساختیه که قراره توی فازهای بعدی قابلیت‌های جدیدی بهش اضافه کنیم.

۳ راه‌اندازی محیط پروژه

قبل از اینکه وارد پیاده‌سازی بشین، باید محیطی داشته باشین که همه‌مون بتونیم کد رو توی شرایط یکسان اجرا و تست کنیم. برای همین قرار نیست روی هر سیستم‌عامل، کتابخونه‌ها رو جداگانه نصب کنین. کدهای پروژه داخل یک کانتینر Ubuntu 24.04 ساخته و اجرا می‌شن، اما Ollama روی سیستم اصلی خودتون اجرا می‌شه.

۱.۳. چیزهایی که باید نصب داشته باشین

روی سیستم خودتون این ابزارها رو نصب کنین: Git، GitHub CLI (gh)، Docker Desktop، Visual Studio Code و افزونه‌ی Dev Containers، و Ollama. لینک‌های رسمی نصب هم این‌ها هستن: Git، GitHub CLI، Docker Desktop، VS Code و Ollama. برای Docker باید حالت Linux Containers فعال باشه؛ روی ویندوز معمولاً از WSL 2 استفاده می‌کنیم. مدل مورد استفاده‌ی این فاز هم qwen2.5:0.5b هست.

> **دو محیط متفاوت داریم:** دستورهای Git و gh برای گرفتن پروژه رو توی ترمینال اصلی کامپیوترتون اجرا می‌کنین. بعد از باز کردن پروژه داخل Dev Container، دستورهای make، تست‌ها و اجرای ECHO-9 رو در ترمینال Ubuntu می‌زنین. نصب و اجرای Ollama هم روی سیستم اصلیه، نه داخل کانتینر.

۲.۳. گرفتن پروژه با Git

برای تحویل پروژه از Git و GitHub استفاده می‌کنیم. هر گروه یک repo خصوصی توی Organization درس خواهد داشت و اعضای گروه فقط روی همون مخزن کار می‌کنن. نباید کدهای گروهتون رو برای بقیه‌ی گروه‌ها قابل مشاهده کنین.

اول باید دعوت عضویت در Organization رو که از طرف تیم حل تمرین دریافت می‌کنین بپذیرین. فقط یک نفر از هر گروه مسئول ساخت مخزن می‌شه؛ عضو دیگه بعداً به همون مخزن اضافه می‌شه. دقت کنین که Organization با مخزن اصلی پروژه یکی نیست؛ آدرس مخزن اصلی پروژه اینجاست.

قبل از شروع، از **ترمینال اصلی سیستم** وارد حساب GitHub بشین:

host terminal - GitHub login

```
git --version
gh --version
gh auth login --web --git-protocol https
gh auth setup-git
gh auth status
```

حالا شماره‌ی گروهتون رو جای 07 دستور اول یک مخزن خصوصی **خالی** می‌سازه و دستور بعدی پروژه‌ی اصلی درس رو clone می‌کنه. در این مرحله Fork نمی‌زنیم.

host terminal - team repository and clone

```
# Replace 07 before running.

TEAM=07
REPO="echo9-team-$TEAM"

gh repo create "fum-os-1405/$REPO" --private

git -c core.autocrlf=input clone \
  https://github.com/HaniehMalekjafarian/ECHO9-Station.git "$REPO"
cd "$REPO"
git config core.autocrlf input

# Course repository = upstream; team repository = origin.
git remote rename origin upstream
```

```
git remote add origin "https://github.com/fum-os-1405/$REPO.git"
git remote -v
git push -u origin main
gh repo set-default origin
```

از اینجا به بعد دو remote دارین: upstream به مخزن اصلی درس اشاره می‌کنه و origin به مخزن خصوصی خودتون. کدهای گروه رو فقط به origin می‌فرستین؛ تغییرهای فازهای بعدی رو هم از upstream می‌گیرین. با این روش تاریخچه‌ی پروژه‌ی شما از همون ابتدا به تاریخچه‌ی مخزن درس متصله.

مهم: سازنده‌ی مخزن باید از صفحه‌ی GitHub وارد Settings > Collaborators & teams بشه و عضو دیگه‌ی گروه رو با نقش Write اضافه کنه. شخص دیگری رو بدون هماهنگی وارد مخزن نکنین. حتماً بررسی کنین مخزن با وضعیت Private ساخته شده باشه.

قبل از اولین commit، اطلاعات خودتون رو روی همین مخزن تنظیم کنین:

host terminal - identify commits

```
git config user.name "Your Name"
git config user.email "your-github-email@example.com"
```

> **نکته‌ی کاربران ویندوز**
>
> فایل‌های shell باید با پایان خط LF باقی بمونن؛ به همین خاطر موقع clone از core.autocrlf=input استفاده کردیم. اگر demo/run_demo.sh با CRLF ذخیره بشه، ممکنه داخل کانتینر با خطای pipefail روبه‌رو بشین.

۳.۳. ورود به محیط Ubuntu و ساخت پروژه

Docker Desktop رو باز کنین و مطمئن بشین در حال اجراست. پوشه‌ی echo9-team-07 رو در VS Code باز کنین. از Command Palette گزینه‌ی Dev Containers: Reopen in Container رو انتخاب کنین. بار اول ممکنه ساخت محیط و گرفتن بسته‌های مورد نیاز بیشتر طول بکشه. بعد از باز شدن ترمینال Ubuntu، این دستورها رو اجرا کنین:

Ubuntu Dev Container

```
cd /workspace
make doctor
make clean && make
```

اگه ساخت پروژه موفق بود، یعنی ابزارهای لازم رو درست راه‌اندازی کردین. اما یادتون باشه کد اولیه عمداً کامل نیست.

۴.۳. راه‌اندازی Ollama و ایستگاه

برنامه رو روی سیستم اصلی نصب و اجرا کنین، بعد توی **ترمینال اصلی سیستم** مدل رو بگیرین:

host terminal - Ollama

```
ollama pull qwen2.5:0.5b
ollama list
curl http://localhost:11434/api/tags
```

حالا از داخل کانتینر، ارتباط با Ollama سیستم اصلی رو بررسی کنین:

Ubuntu Dev Container - connectivity check

```
python3 -c "from urllib.request import urlopen; \
print(urlopen('http://host.docker.internal:11434/api/tags', \
timeout=8).status)"
```

وقتی بخش‌های مربوط به خودتون رو کامل کردین، برای اجرای واقعی ایستگاه داخل یک ترمینال Ubuntu بنویسین:

Ubuntu Dev Container - live station

```
cd /workspace
OLLAMA_MODEL=qwen2.5:0.5b bash demo/run_demo.sh ollama
```

این دستور خودش هم سرور C و هم واسطه‌ی Python رو اجرا می‌کنه؛ پس هم‌زمان make run نزنین. پورت 8765 رو توی بخش Ports در VS Code باز کنین و رابط ایستگاه رو ببینین: / برای Mission Control، /station برای نمای ایستگاه، /workbench برای آزمایش پیام‌ها و /processes برای وضعیت و چرخه‌ی عمر ایجنت‌ها.

۵.۳. چطور بفهمیم کارمون درسته؟

تست‌ها رو داخل کانتینر اجرا کنین:

Ubuntu Dev Container - public tests

```
python3 tests/test_process_manager_unit.py
make test
```

نسخه‌ی فعلی **۱۳ مجموعه تست عمومی** داره. قبل از اینکه قسمت‌های TODO رو کامل کنین، رد شدن تست‌ها طبیعیه. این تست‌ها برای ثابت نگه داشتن نتیجه‌ها از یک سرویس آزمایشی سازگار با Ollama استفاده می‌کنن؛ با این حال نصب Ollama و اجرای واقعی ایستگاه برای این پروژه الزامیه.

۴ تحویل با Git

۱.۴. شاخه‌ها و Pull Request

در این فاز دو بخش اصلی دارین: src/main.c و src/process_manager.c. بهتره تغییرهای مربوط به هر بخش رو روی یک branch جدا انجام بدین و قبل از وارد کردن اون‌ها به main، کد رو مرور کنین. این‌طوری هم تاریخچه‌ی کارتون مشخص می‌مونه و هم احتمال اینکه تغییرها رو ناخواسته از بین ببرین کمتره.

مثلاً برای پیاده‌سازی ProcessManager، داخل **ترمینال Ubuntu** این دستورها رو اجرا کنین:

Ubuntu Dev Container - implement and commit

```
git switch -c feat/pcb-lifecycle
# Complete the student TODOs in src/process_manager.c.
make && python3 tests/test_process_manager_unit.py

git add src/process_manager.c
git commit -m "feat(p1): implement PCB lifecycle"
```

بعد توی **ترمینال اصلی کامپیوتر** و در همون پوشه‌ی پروژه، شاخه‌تون رو منتشر کنین و یک Pull Request به مخزن گروه بسازین. چون قبلاً gh repo set-default origin رو اجرا کردیم، gh مخزن اصلی درس رو به‌اشتباه به‌عنوان مقصد انتخاب نمی‌کنه.

host terminal - publish a team PR

```
cd echo9-team-07
git push -u origin feat/pcb-lifecycle

gh pr create --base main --head feat/pcb-lifecycle \
  --title "Implement ProcessManager" \
  --body "PCB registration and lifecycle handling."
```

در روز ارائه تاریخچه pull requestها بررسی میشه و نقش هر عضو تیم روی پیشرفت پروژه بررسی و نمره‌دهی میشه. بنابراین از همین اول مراحل پیشرفت پروژتون رو ثبت کنین.

ترجیحا main رو همیشه سبز نگه دارین. از طریق صفحه‌ی Pull Request، تغییرها رو بررسی کنین و بعد از رفع مشکل‌های احتمالی با main ادغام کنین. اگه دوست داشتین، برای بخش C Server هم از شاخه‌ی feat/m1-socket استفاده کنین. **مستقیم روی مخزن اصلی درس push نکنین.** این روش کار رو حتی اگه GitHub به‌صورت اجباری برای main قانون محافظت نداشته باشه رعایت کنین.

۲.۴. تست خودکار با GitHub Actions

توی مخزنی که clone کردین، فایل github/workflows/ci-p1.yml از قبل وجود داره. با هر push و Pull Request، سه job روی Ubuntu 24.04 اجرا می‌شن: build برای کامپایل، pcb برای تست مستقل ProcessManager و all-tests برای اجرای **۱۳ مجموعه تست عمومی**. نتیجه رو توی تب Actions مخزن گروه ببینین.

در شروع پروژه رد شدن تست‌ها طبیعیه، چون بخشی از کد هنوز کامل نشده. اما قبل از تحویل باید تست‌ها رو اجرا کنین و خطاها رو برطرف کنین. در روز ارائه، هر دو عضو گروه باید بتونن درباره‌ی کد و تصمیم‌های فنی توضیح بدن.

host terminal - inspect CI

```
gh run list
gh run view --log-failed
```

بعد از ادغام تغییرها در main، روی نسخه‌ی نهایی تست بگیرین. برای ثبت تحویل، تگ p1-final رو روی همون commit بسازین:

host terminal - prepare final version

```
git switch main
git pull --ff-only origin main
```

Ubuntu Dev Container - final tests

```
make test
```

host terminal - submit

```
git tag -a p1-final -m "P1 final submission"
git push origin p1-final
git rev-parse HEAD
```

چیزهایی که باید تحویل بدین

لینک مخزن خصوصی گروه داخل Organization درس، نام تگ p1-final و شناسه‌ی commit نهایی رو تحویل بدین. داخل مخزن باید src/main.c، src/process_manager.c، P1_StudentID1_StudentID2.pdf و AI_USE.md موجود باشن. فایل‌های build/، وزن‌های مدل و فایل‌های خصوصی تیم حل تمرین نباید وارد مخزن بشن. به‌جز قسمت‌هایی که برای پیاده‌سازی شما مشخص شده، فایل‌های آماده‌ی پروژه رو تغییر ندین.

۳.۴. گرفتن تغییرهای فاز بعد

این پنج فاز پروژه‌های مستقل نیستن. وقتی فاز بعد منتشر شد، از upstream تغییرهای جدید رو وارد همون مخزن گروه می‌کنین:

host terminal - after the next phase is released

```
git switch main
git fetch upstream --tags
git merge upstream/main
git push origin main
```

اگر تغییری در فایل‌های مشترک ایجاد شده باشه، ممکنه merge conflict داشته باشین. اون رو بررسی و حل کنین؛ کد قبلی خودتون رو فقط برای اینکه ادغام سریع‌تر تموم بشه کنار نذارین.

۵ ایستگاه واقعی و رابط‌های کاربری

تا اینجا درباره‌ی معماری داخلی AgentOS و بخش‌هایی که باید پیاده‌سازی کنین صحبت کردیم. اما این سیستم فقط چند فایل C نیست که جدا از بقیه اجرا بشه.

برای این پروژه، یک ایستگاه واقعی به اسم ECHO-9 و چند رابط کاربری مختلف در اختیارتون قرار می‌گیره تا بتونین سیستم خودتون رو در عمل ببینین، باهاش کار کنین و رفتار بخش‌های مختلفش رو مشاهده کنین.

این رابط‌ها صرفاً نمایشی نیستن. وقتی از اون‌ها استفاده می‌کنین، درخواست‌ها واقعاً از طریق رابط وب و واسطه‌ی پایتونی به سرور C شما می‌رسن، از Unix Socket عبور می‌کنن، وارد AgentOS می‌شن و در نهایت به مدل زبانی محلی می‌رسن. به همین خاطر، این بخش‌ها فقط برای دمو نیستن؛ بلکه یکی از بهترین راه‌ها برای دیدن نتیجه‌ی واقعی پیاده‌سازی شما هستن.

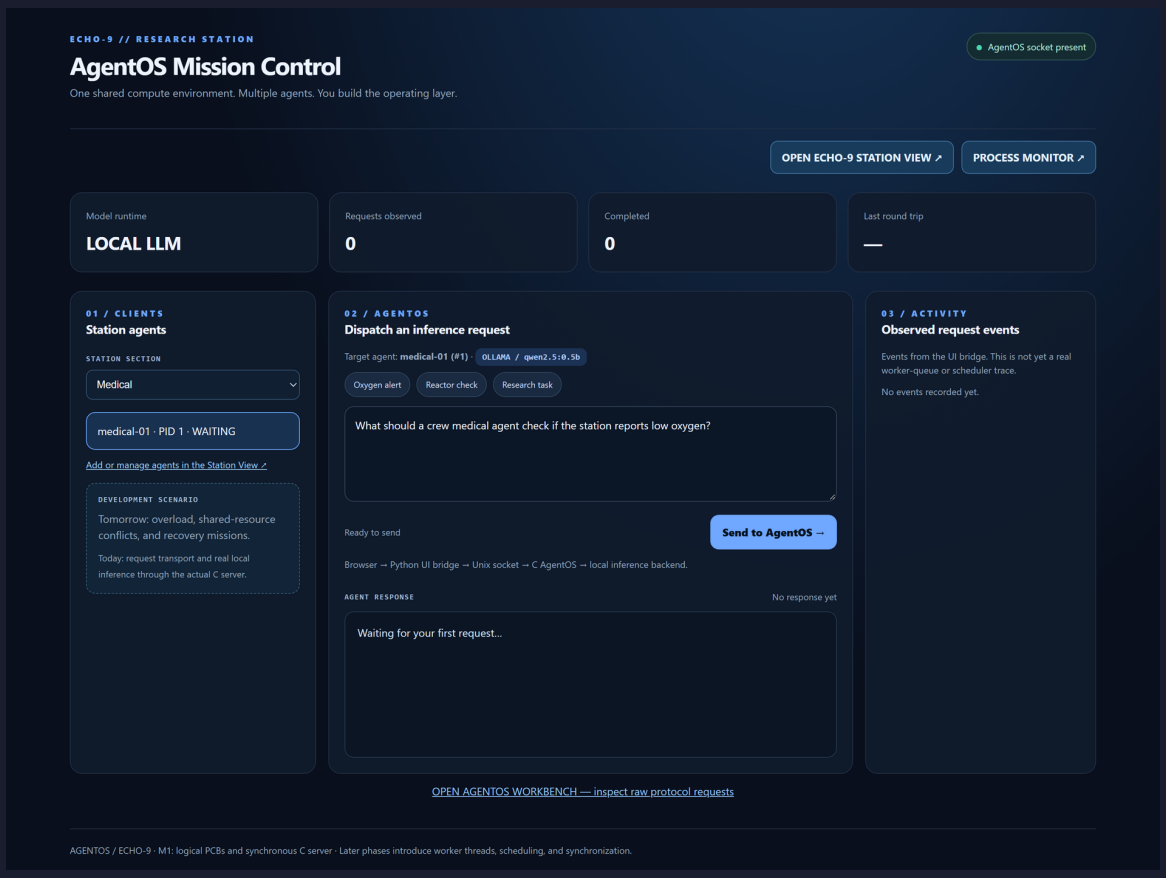
۱.۵. Mission Control

Mission Control اصلی‌ترین رابط برای ارسال درخواست‌های عادی به AgentOS هست.

در این صفحه می‌تونین یکی از ایجنت‌های ایستگاه رو انتخاب کنین، برای اون درخواست بفرستین و پاسخ تولیدشده توسط مدل رو ببینین. هم‌زمان اطلاعاتی مثل تعداد درخواست‌ها، زمان رفت‌وبرگشت و رویدادهای مشاهده‌شده هم نمایش داده می‌شن.

این بخش به شما کمک می‌کنه ببینین که آیا مسیر اصلی ارسال و دریافت درخواست درست کار می‌کنه یا نه. اگر سرور شما نتونه درخواست رو درست دریافت کنه، تحویل بده یا پاسخ رو کامل برگردونه، اولین جایی که این مشکل خودش

رو نشون می‌ده معمولاً همین صفحه است.



شکل ۲: Mission Control: ارسال درخواست از طرف ایجنت‌ها و مشاهده‌ی پاسخ مدل.

۲.۵ Process Monitor

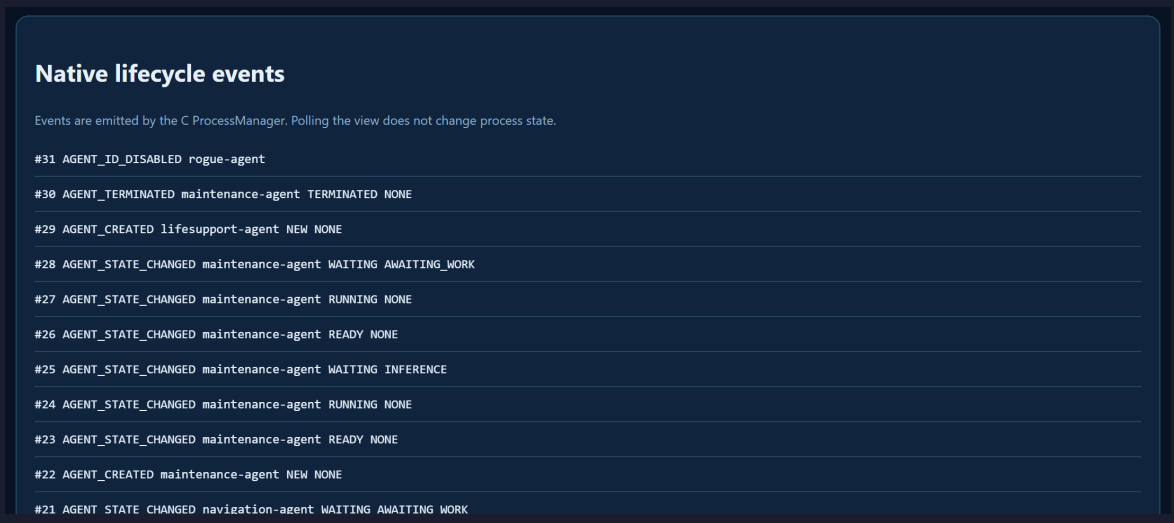
Process Monitor برای مشاهده‌ی وضعیت داخلی ایجنت‌ها و PCB های منطقی طراحی شده.

توی این بخش می‌تونین لیست ایجنت‌های فعال، ایجنت‌های خاتمه‌یافته، وضعیت هر PCB، دلیل انتظار یا خروج و همچنین عملیات مربوط به چرخه‌ی عمر مثل terminate و reap رو ببینین.



شکل ۳: نمای PCB Roster: مشاهده‌ی PCB های فعال و خاتمه‌یافته و عملیات مربوط به چرخه‌ی عمر.

علاوه بر این، نمای Native lifecycle events رویدادهای ثبت‌شده توسط ProcessManager شما رو نشون می‌ده. این قسمت خیلی مهمه، چون به شما کمک می‌کنه تغییر وضعیت ایجنت‌ها رو نه فقط در سطح خروجی نهایی، بلکه به‌صورت مرحله‌به‌مرحله ببینین. اگر در پیاده‌سازی ProcessManager اشتباهی وجود داشته باشه، معمولاً اینجا خیلی سریع خودش رو نشون می‌ده. مثلاً تغییر وضعیت نادرست، جمع‌آوری اشتباه، یا ثبت نشدن درست یک هویت جدید.



شکل ۴: نمای Native lifecycle events: دنباله‌ی رویدادهای ثبت‌شده توسط ProcessManager.

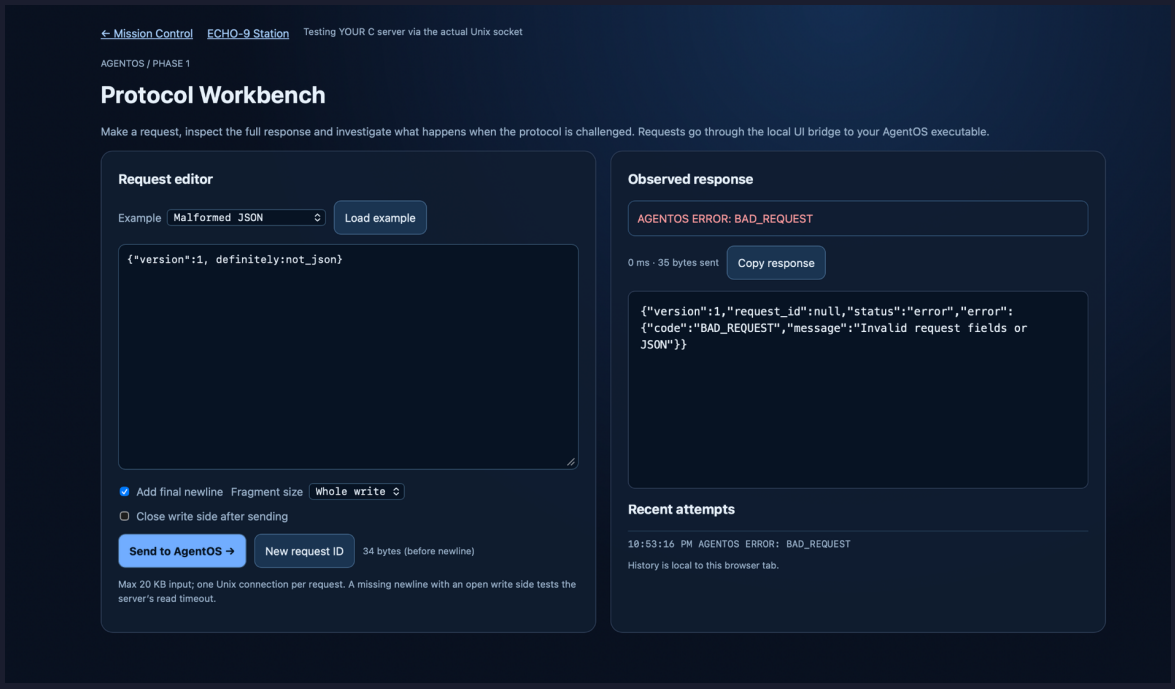
۳.۵. Protocol Workbench

Protocol Workbench جاییه که می‌تونین رفتار سرور خودتون رو در برابر درخواست‌های غیرعادی یا خراب بررسی کنین.

در این بخش می‌شه درخواست‌های مختلفی ساخت، متن خام پیام رو تغییر داد و نتیجه‌ی پاسخ سرور رو مستقیماً دید. برای مثال، می‌تونین ببینین اگر JSON نامعتبر باشه، یک فیلد ضروری وجود نداشته باشه، یا پیام ناقص فرستاده بشه، سرور شما چه رفتاری نشون می‌ده.

این رابط برای تست مسیر اصلی استفاده نمی‌شه، بلکه بیشتر به درد فهمیدن و آزمایش کردن مرزهای پروتکل می‌خوره.

به کمک این بخش، می‌تونین خیلی واضح ببینین آیا سرورتون در برخورد با ورودی نامعتبر، رفتار درست و قابل پیش‌بینی داره یا نه.



شکل ۵: Protocol Workbench: آزمایش رفتار سرور در برابر درخواست‌های ناقص، نامعتبر یا دست‌کاری‌شده.

۴.۵. Live Station View

Live Station View یک نمای زنده از خود ایستگاهه.

در این بخش، ایجنت‌های مختلف رو داخل محیط ECHO-9 می‌بینین و هم‌زمان وضعیت کلی ایستگاه، درخواست‌های در حال اجرا و فعالیت مشاهده‌شده‌ی AgentOS نمایش داده می‌شه.

این نما قرار نیست جایگزین ابزارهای فنی‌تر مثل Process Monitor یا Workbench بشه؛ بلکه کمک می‌کنه ارتباط بین چیزی که در سطح داخلی سیستم اتفاق می‌افته و چیزی که در سطح ایستگاه دیده می‌شه رو بهتر درک کنین.

به بیان ساده‌تر، اینجا می‌تونین نتیجه‌ی زنده‌ی درخواست‌هایی رو ببینین که از طریق سیستم شما در حال عبور هستن.



شکل ۶: Live Station View: نمای زنده‌ی ایستگاه و مشاهده‌ی فعالیت ایجنت‌ها در محیط ECHO-9.

۵.۵. چرا این رابط‌ها مهمن؟

این رابط‌ها چند کاربرد مهم دارن:

- کمک می‌کنن فقط به خروجی تست‌ها اکتفا نکنین و رفتار واقعی سیستم رو هم ببینین.
- اشکال‌زدایی رو راحت‌تر می‌کنن. خیلی وقت‌ها یک خطا از روی کد یا تست مبهمه، اما وقتی همون رفتار رو در Mission Control یا Process Monitor می‌بینین، مسئله خیلی واضح‌تر می‌شه.
- برای گزارش پروژه و روز ارائه هم مهمن. شما باید بتونین نشون بدین که پیاده‌سازی‌تون فقط روی کاغذ درست نیست، بلکه در محیط واقعی ایستگاه هم کار می‌کنه.

در نهایت، یادتون باشه که این رابط‌ها از قبل برای شما آماده شدن؛ چیزی که باید درست کار کنه، هسته‌ی واقعی سیستم شماست. به بیان دیگه، قرار نیست شما UI ها رو پیاده‌سازی کنین؛ قرار است سروری بنویسین که این UI ها بتوانند به آن تکیه کنند.

۶ چیزهایی که باید تحویل بدین

در پایان این پروژه، فقط کدی که نوشتین رو بررسی نمی‌کنیم. می‌خوایم بدونیم چطور به پیاده‌سازی نهایی رسیدین، چه تصمیم‌هایی گرفتین، با چه مشکلاتی مواجه شدین و چطور مطمئن شدین که سیستم درست کار می‌کنه. برای همین تحویل این فاز سه بخش داره.

۱.۶ مخزن خصوصی گروه

کدهای نهایی باید داخل همون repo خصوصی گروهتون در Organization درس قرار داشته باشن.

قبل از تحویل، مطمئن بشین تغییرهای هر دو عضو گروه روی شاخه‌ی main ادغام شدن، تست‌ها رو روی آخرین نسخه اجرا کردین و تگ p1-final رو روی همون نسخه ساختین.

اطلاعاتی که باید تحویل بدین شامل لینک مخزن گروه، نام تگ نهایی و Commit SHA مربوط به اون تگه.

توی مخزن باید دو فایل `src/main.c` و `src/process_manager.c` کامل شده باشن. فایل گزارش و اظهارنامه‌ی استفاده از هوش مصنوعی هم باید همراه کدها توی همین مخزن قرار بگیرن.

۲.۶ گزارش روند انجام پروژه

علاوه بر کد، هر گروه باید یک گزارش با فرمت PDF و نام `P1_StudentID1_StudentID2.pdf` آماده کنه و داخل ریشه‌ی مخزن قرار بده. این گزارش قرار نیست فقط توضیح بده هر تابع چه کاری انجام می‌ده. چیزی که بیشتر برامون اهمیت داره، مسیریه که برای انجام پروژه طی کردین. توی گزارش به این موارد بپردازین:

- **معرفی گروه و تقسیم کار:** مشخص کنین هر عضو مسئول چه بخش‌هایی بوده و در قسمت‌های مشترک چطور با هم همکاری کردین.
- **تصمیم‌های پیاده‌سازی:** خیلی مختصر توضیح بدین برای بخش‌های C Server و ProcessManager چه ساختاری انتخاب کردین و چرا. مثلاً اطلاعات PCB ها رو چطور نگه داشتین و مدیریت حافظه رو چطور انجام دادین.
- **مشکلات و راه‌حل‌ها:** چه خطاها یا رفتارهای غیرمنتظره‌ای دیدین؟ چطور علتشون رو پیدا کردین و چه تغییری باعث شد مشکل برطرف بشه؟ سعی کنین حداقل یک نمونه‌ی واقعی از روند اشکال‌زدایی خودتون رو توضیح بدین.
- **تست و بررسی صحت برنامه:** دستورهایی که برای تست اجرا کردین، نتیجه‌ی تست‌های عمومی و وضعیت GitHub Actions رو گزارش کنین. اگه تستی هنوز شکست می‌خوره، مشخص کنین کدوم تسته و به چه دلیلی.
- **اجرای واقعی ایستگاه:** نتیجه‌ی اتصال به Ollama و اجرای ECHO-9 رو نشون بدین. حداقل یک درخواست واقعی و یک نمونه از تغییر وضعیت ایجنت‌ها، از ایجاد تا خاتمه و جمع‌آوری نهایی، رو بررسی کنین.

می‌تونین از تصویرهای Mission Control، Workbench و Process Monitor برای نشون دادن نتیجه‌ی کارتون استفاده کنین. اما صرفاً گذاشتن اسکرین‌شات کافی نیست؛ توضیح بدین هر تصویر چه چیزی رو نشون می‌ده و چرا نتیجه‌ای که گرفتین درسته.

گزارش باید بر اساس کاری باشه که واقعاً انجام دادین. لازم نیست همه‌ی مراحل بدون مشکل پیش رفته باشن. حتی ممکنه توضیح یک خطا و روشی که برای پیدا کردن علتش استفاده کردین، از چند صفحه تصویر اجرای موفق ارزشمندتر باشه.

۳.۶. اظهارنامه‌ی استفاده از هوش مصنوعی

آخرین فایل موردنیاز، `AI_USE.md` هست. توی این فایل باید مشخص کنین از چه ابزارهای هوش مصنوعی استفاده کردین، برای چه کاری ازشون کمک گرفتین و لینک گفت‌وگوها رو قرار بدین. جزئیات استفاده‌های مجاز و غیرمجاز رو توی بخش بعد توضیح می‌دیم.

۷ سیاست استفاده از هوش مصنوعی

این پروژه خودش بر پایه‌ی ایجنت‌ها و هوش مصنوعی طراحی شده، پس طبیعتاً قرار نیست استفاده از هوش مصنوعی برای یادگیری رو ممنوع کنیم. اما بین اینکه از هوش مصنوعی برای **فهمیدن یک مسئله** کمک بگیرین و اینکه **حل کردن اون مسئله رو بهش بسپرین** تفاوت وجود داره.

هدف این پروژه اینه که خودتون با مفاهیم سیستم‌عامل درگیر بشین، کد بنویسین، اشتباه کنین، اشکال‌زدایی کنین و در نهایت متوجه بشین سیستمی که ساختین چطور و چرا کار می‌کنه.

برای همین، استفاده از ابزارهای هوش مصنوعی توی این پروژه قوانین مشخصی داره.

۱.۷. استفاده‌های مجاز

می‌تونین از ابزارهای هوش مصنوعی برای موارد زیر کمک بگیرین:

- **یادگیری مفاهیم:** پرسیدن سؤال درباره‌ی مفاهیمی مثل Socket، Blocking I/O، Buffer، Timeout، PCB و چرخه‌ی عمر فرآیندها.
- **درک خطاها و ابزارها:** درخواست توضیح درباره‌ی پیام‌های کامپایلر، خطاهای سیستم‌عامل، ابزارهای لینوکس و مستندات توابع کتابخانه‌ای.
- **بررسی ایده‌ها و مفاهیم تست:** گفت‌وگو درباره‌ی روش‌های کلی طراحی، ایده‌های تست و اینکه چرا یک رفتار از نظر مفهومی درست یا نادرسته.
- **کمک در مستندسازی:** کمک برای مرتب کردن توضیحات، اصلاح نگارش و بهتر نوشتن متن‌هایی که خودتون تهیه کردین. البته تحلیل فنی، تصمیم‌های پیاده‌سازی و نتایج آزمایش‌ها باید حاصل کار خودتون باشن. مسئولیت صحت تمام مطالب هم به عهده‌ی شماست.

۲.۷. استفاده‌های غیرمجاز

موارد زیر توی این پروژه مجاز نیستن:

- **تولید کد پروژه:** درخواست تولید یا تکمیل کدهای TODO،نوشتن توابع `src/main.c` یا `src/process_manager.c` و گرفتن کد یا Patch آماده برای حل پروژه.
- **سپردن اشکال‌زدایی به AI:** فرستادن کد پیاده‌سازی خودتون به AI با هدف دریافت نسخه‌ی اصلاح‌شده، بازنویسی توابع یا گرفتن راه‌حل مستقیم برای تست‌هایی که شکست خوردن.
- **استفاده از ابزارهای کدنویسی خودکار:** استفاده از AI Coding Agents، Agent Mode یا ابزارهایی که به‌جای

شما فایل‌های پروژه رو بررسی کنن، تغییر بدن، تست اجرا کنن یا پیاده‌سازی رو جلو ببرن.

- **سپردن تحلیل و گزارش به AI:** درخواست تولید تحلیل فنی، توضیح تصمیم‌هایی که خودتون نگرفتین یا ساختن نتایج و آزمایش‌هایی که واقعاً انجام ندادین.

> **قاعده‌ی اصلی**
>
> هوش مصنوعی می‌تونه بهتون کمک کنه مسئله رو **بفهمین**، اما نباید مسئله رو **به‌جای شما حل یا کدنویسی کنه**. اگه درباره‌ی مجاز بودن یک نوع استفاده مطمئن نیستین، قبل از انجامش از تیم حل تمرین سؤال کنین.

۳.۷. شفافیت استفاده از هوش مصنوعی

استفاده از AI به‌خودی‌خود تخلف محسوب نمی‌شه، اما استفاده از اون باید شفاف باشه.

هر استفاده‌ای که در رابطه با این فاز از ابزارهای هوش مصنوعی داشتین باید ثبت بشه؛ حتی اگه فقط برای فهمیدن یک مفهوم یا معنی یک خطای کامپایلر بوده باشه.

برای این کار، فایل `AI_USE.md` رو داخل ریشه‌ی مخزن گروه قرار بدین. برای هر گفت‌وگوی مرتبط، موارد زیر رو ثبت کنین:

- نام ابزار هوش مصنوعی که ازش استفاده کردین.
- اینکه کدوم عضو گروه از ابزار استفاده کرده.
- موضوع سؤال و هدف استفاده از ابزار.
- توضیح کوتاهی درباره‌ی اینکه چه کمکی گرفتین و چه چیزی از اون گفت‌وگو یاد گرفتین.
- **لینک اشتراک‌گذاری گفت‌وگوی مرتبط** که برای تیم حل تمرین قابل مشاهده باشه.

لینک‌ها باید موقع بررسی توسط تیم حل تمرین قابل باز شدن باشن. صرفاً نوشتن اینکه «از ChatGPT استفاده کردیم» یا «برای دیباگ از Claude کمک گرفتیم» کافی نیست.

بهتره برای سؤال‌های مربوط به پروژه، گفت‌وگوهای جداگانه بسازین تا بتونین بدون به اشتراک گذاشتن مکالمات شخصی، لینک‌های مربوط به پروژه رو ارائه بدین.

اگه از ابزاری استفاده می‌کنین که امکان ایجاد لینک قابل مشاهده برای گفت‌وگوهاش رو نداره، قبل از استفاده برای پروژه با تیم حل تمرین هماهنگ کنین.

۴.۷. ارزیابی و روز ارائه

در روز ارائه، هر دو عضو گروه باید بتونن درباره‌ی تمام قسمت‌های کدی که تحویل دادن توضیح بدن و به سؤال‌های مفهومی و کدنویسی مربوط به پروژه پاسخ بدن.

ممکنه ازتون بخوایم بخشی از کد رو توضیح بدین، رفتار اون رو در یک شرایط جدید پیش‌بینی کنین یا درباره‌ی علت انتخاب یک روش پیاده‌سازی استدلال کنین.

اینکه از AI برای یادگرفتن یک مفهوم کمک گرفته باشین مشکلی نداره، اما باید بتونین نشون بدین که خودتون اون مفهوم رو یاد گرفتین.